

This process—checking the first elements of the left and right cabinets and popping the appropriate one into the new cabinet—needs to continue as long as both of the cabinets still have at least one file. That’s why we will nest these `if` statements inside a `while` loop that checks the minimum length of `left` and `right`. As long as both `left` and `right` contain at least one file, it will continue its process:

```
while(min(len(left),len(right)) > 0):
    if left[0] > right[0]:
        to_insert = right.pop(0)
        newcabinet.append(to_insert)
    elif left[0] <= right[0]:
        to_insert = left.pop(0)
        newcabinet.append(to_insert)
```

Our `while` loop will stop executing as soon as either `left` or `right` runs out of files to insert. At that point, if `left` is empty, we’ll insert all the files in `right` at the end of the new cabinet in their current order, and vice versa. We can accomplish that final insertion as follows:

```
if(len(left) > 0):
    for i in left:
        newcabinet.append(i)

if(len(right) > 0):
    for i in right:
        newcabinet.append(i)
```

Finally, we combine all of those snippets into our final merging algorithm in Python as shown in [Listing 4-4](#).

```
def merging(left,right):
    newcabinet = []
    while(min(len(left),len(right)) > 0):
        if left[0] > right[0]:
            to_insert = right.pop(0)
            newcabinet.append(to_insert)
        elif left[0] <= right[0]:
            to_insert = left.pop(0)
            newcabinet.append(to_insert)
    if(len(left) > 0):
        for i in left:
            newcabinet.append(i)
```